

Claude e Claude Code in pratica

Dal prompting agli agenti

Simone Scannapieco

Corso base per Tecnodata Trentina S.r.l., Trento, Italia

Modulo 6 — Projects e Context Engineering

CLAUDE.md: istruzioni che si caricano sempre

CLAUDE.md: istruzioni che si caricano sempre

Lo scrivete voi; Claude lo legge all'inizio di ogni sessione

- Nel contesto di Claude Code ci sono la conversazione, i file letti, gli output dei comandi, le istruzioni di sistema; e il `CLAUDE.md`, la parte che **scrivete voi**. Dal blog di Claude: «Your CLAUDE.md file loads into the context at the start of every session, so each line in it is part of what every turn resends.»
- Accanto c'è l'**auto memory**, la memoria automatica del Modulo 4, che invece scrive Claude: «notes Claude writes itself based on your corrections and preferences»
- Dove arriva, dalla documentazione: «CLAUDE.md content is delivered as a **user message** after the system prompt, not as part of the system prompt itself.»
- Claude lo legge e cerca di seguirlo, ma «there's no guarantee of strict compliance, especially for vague or conflicting instructions.» Le regole delle impostazioni le applica il client; le istruzioni del `CLAUDE.md` «are not a hard enforcement layer»
- «Claude treats them as context, not enforced configuration.» Ciò che deve valere ogni volta lo esegue Claude Code stesso: «This can be a hook, a deny rule, or a managed setting.» Hook e regole deny sono del Modulo 4

Un `CLAUDE.md` è contesto che si carica sempre, non una regola che si impone.

Quattro posti, un ordine

Dove sta un `CLAUDE.md`, in che ordine si legge, e quando si carica

Un `CLAUDE.md` può stare in **quattro posti**, dal più generale al più vicino:

posto	file	che cosa
gestito	`/etc/claude-code/CLAUDE.md`, su Linux e WSL	«Organization-wide instructions managed by IT/DevOps»
utente	`~/.claude/CLAUDE.md`	le vostre istruzioni, «User instructions»
progetto	`./CLAUDE.md` o `./.claude/CLAUDE.md`	per chiunque lavori al progetto: comandi di build e di test, standard, convenzioni
locale	`./CLAUDE.local.md`	«private per-project preferences that shouldn't be checked into version control»

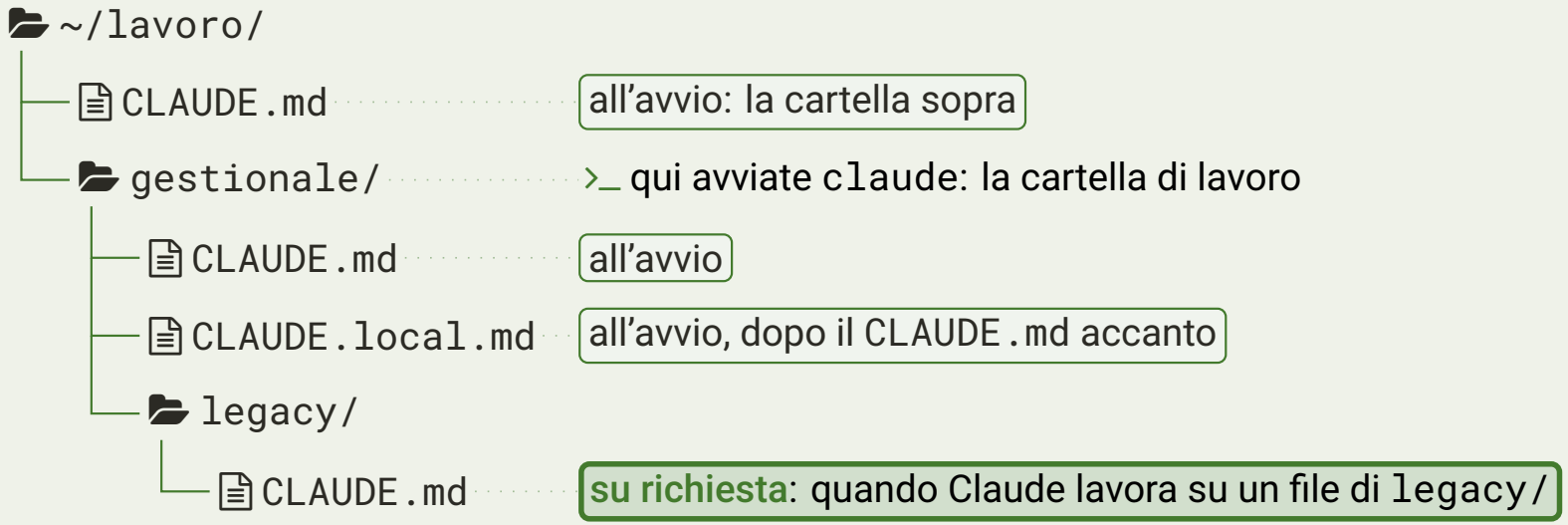
Dalla pagina «How Claude remembers your project»

Non si sostituiscono, **si sommano**: «All discovered files are concatenated into context rather than overriding each other.»

L'ordine: dalla radice del filesystem fino alla cartella di lavoro, e «instructions closer to where you launched Claude are read last». In ogni cartella il `CLAUDE.local.md` viene dopo il `CLAUDE.md`: «your personal notes are the last thing Claude reads at that level»

Quattro posti, un ordine

Dove sta un `CLAUDE.md`, in che ordine si legge, e quando si carica



Uno schema: i nomi delle cartelle sono d'esempio

- Dalla documentazione: «CLAUDE.md and CLAUDE.local.md files in the directory hierarchy above the working directory are loaded at launch. Files in subdirectories **load on demand**.»
- Su richiesta vuol dire: quando Claude **legge, scrive o modifica** un file di quella sottocartella; e conta come lettura anche un `cat` o un `head` su un solo file
- E `AGENTS.md`? Claude Code può leggerlo come istruzioni di progetto, così un repository preparato per altri agenti funziona senza un `CLAUDE.md`; con tutti e due, nella cartella di lavoro o sopra, di default: «Your CLAUDE.md files only»

Import, commenti, regole

La sintassi, in un `CLAUDE.md` e in una regola

CLAUDE.md, un esempio del corso

```
# Istruzioni del progetto

- Rispondi in italiano; i termini tecnici restano in inglese.

<!--
Per chi mantiene il file: questo commento non arriva a Claude.
-->

- Glossario: @docs/glossario.md
- Da non importare: `@docs/escluso.md`
```

- `@percorso` **importa**: «CLAUDE.md files can import additional files using @path/to/import syntax.» Relativo al file che importa; fino a quattro passaggi
- Fra backtick si **cita**, non si importa: «To mention a path in your CLAUDE.md without importing it, wrap it in backticks»
- **Non fa risparmiare**: «Imports help you organize a long file but don’t reduce its context cost»
- Il commento a blocco **non arriva** a Claude: «Block-level HTML comments … are stripped before the content is injected into Claude’s context.»

Import, commenti, regole

La sintassi, in un CLAUDE.md e in una regola

`.claude/rules/php.md`, un esempio del corso

```
---
paths:
  - "**/*.php"
---

# Regola per il codice PHP

- Ogni file PHP nuovo inizia con `declare(strict_types=1);`.
```

- Le **regole** stanno in `.claude/rules/`. Senza `paths`: «Rules without paths frontmatter are loaded at launch with the same priority as .claude/CLAUDE.md.»
- Con `paths`, **solo quando servono**: «Rules can also be scoped to specific file paths, so they only load into context when Claude works with matching files». E lavorare vuol dire: «A path-scoped rule loads when Claude uses the Read, Write, or Edit tool on a matching file.»
- Il formato che la pagina consiglia per il CLAUDE.md, e che il corso usa anche per le regole: «group related instructions under markdown headers and bullets. Organized sections are easier for Claude to follow than dense paragraphs.»

M6-D02 Che cosa entra all’avvio, e che cosa quando serve

Demo del docente in Claude Code: nove parole in codice, in nove posti

- La frase da mettere alla prova, dalla documentazione: «CLAUDE.md and CLAUDE.local.md files in the directory hierarchy above the working directory are loaded at launch. Files in subdirectories load on demand.» E con lei quelle dei due frame precedenti: import, backtick, commento, regole
- Un piccolo progetto, con **nove parole in codice** casuali, nuove a ogni chiamata, in nove posti:

`CLAUDE.md`, una riga	un commento a blocco nel `CLAUDE.md`	`docs/glossario.md`, importato con `@`
`docs/escluso.md`, citato fra backtick	`CLAUDE.local.md`	`.claude/rules/generale.md`, senza `paths`
`.claude/rules/php.md`, con `paths` sui file PHP	`sub/CLAUDE.md`	`sub/nota.txt`, un file di testo qualunque

- Due chiamate di `claude -p`, Sonnet, effort low, ognuna in una copia nuova: `avvio`, **senza strumenti**, chiede di elencare le parole delle istruzioni di progetto; `lettura`, con il **solo Read**, prima legge `src/a.php` e `sub/nota.txt`, poi elenca. Un hook `InstructionsLoaded` scrive una riga per ogni file di istruzioni che si carica, con il motivo

```
m6_projects_context/s5_projects_context/m6_3_claude_md/\
M6-D02_che_cosa_entra/demo/in_classe.sh \
  ~/corso-claude-code/M6-D02
```

- **Indovinate, a voce**: delle nove parole in codice, quante vedrà Claude all’avvio?

M6-D02 Che cosa entra all’avvio, e che cosa quando serve

Demo del docente in Claude Code: nove parole in codice, in nove posti

- **La registrazione del corso**, tre esecuzioni. **La risposta: quattro**, in tutte e tre: la radice, l’import, il file locale e la regola senza `paths`. Il controllo: «radice sì; commento no; importato sì; fra backtick no; locale sì; regola sì; regola PHP no; sub/CLAUDE.md no; file letto no»
- Dopo i due Read, anche la regola con `paths` e `sub/CLAUDE.md`, in tutte e tre. Il registro dell’hook della prima lettura, una riga per caricamento:

motivo	file	da dove
session_start	.claude/rules/generale.md	
session_start	CLAUDE.md	
session_start	CLAUDE.local.md	
include	docs/glossario.md	da CLAUDE.md
path_glob_match	.claude/rules/php.md	trigger src/a.php; globs **/*.php
nested_traversal	sub/CLAUDE.md	trigger sub/nota.txt

lettura, 1: i caricamenti, come il controllo li ha stampati, in tabella

Registrazione del corso, 7 novembre 2026, Claude Code 2.1.319, Sonnet, effort low

M6-D02 Che cosa entra all’avvio, e che cosa quando serve

Demo del docente in Claude Code: nove parole in codice, in nove posti

```
i verdetti, a capo per la slide: «[…]» segna un taglio

All'avvio si caricano CLAUDE.md, CLAUDE.local.md e la regola
  senza paths: confermata 6 su 6
Il file importato con @ si carica con CLAUDE.md: confermata 6 su 6
Un percorso fra backtick non si importa: confermata 6 su 6
Il commento HTML a blocco non entra nel contesto:
  confermata 3 su 3
La regola con paths e il CLAUDE.md di sub/ non si caricano
  all'avvio: confermata 3 su 3
La regola con paths si carica quando Claude legge un file che
  corrisponde: confermata 3 su 3
Il CLAUDE.md di sub/ si carica quando Claude legge un file di
  sub/: confermata 3 su 3
[…]
Ogni garanzia è confermata in ogni esecuzione.
```

- Il commento si decide nella chiamata **senza strumenti**: «When you open a CLAUDE.md file directly with the Read tool, comments remain visible.»
- Come dato: la parola di `sub/nota.txt`, letta con il Read, in **nessuna** risposta, 0 su 3

Registrazione del corso, 7 novembre 2026, Claude Code 2.1.319, Sonnet, effort low

/init, /memory, /context all, l’auto memory

Gli strumenti per scriverlo, aprirlo e vederlo caricato

- `/init`: «Initialize project with a CLAUDE.md guide.» Una prima versione, da rileggere riga per riga con le due domande della documentazione: «Could it find this by reading the code?» e «Would it actually go and look?» «If both are yes, leave it out.» Nel laboratorio del modulo lo fate sul gestionale o sulla rete
- `/memory`: «Edit CLAUDE.md files, enable or disable auto memory, and view auto memory entries»
- `/context all`: «To confirm the file loaded, run /context all in a session and check the list under Memory files.» E: «If one is missing there, Claude can’t see it.» Un `CLAUDE.md` di una sottocartella lì non c’è: «it loads on demand rather than at launch»
- L’**auto memory**: note di quattro tipi, `user`, `feedback`, `project` e `reference`; salta ciò che si ricava dal codice e ciò che i `CLAUDE.md` dicono già; di `MEMORY.md`, all’inizio di ogni conversazione, entrano le prime 200 righe o i primi 25KB. E se chiedete a Claude di ricordare qualcosa, finisce lì: «To add instructions to CLAUDE.md instead, ask Claude directly»

- Nell’esercitazione che segue, «Che cosa vede Claude», lo verificate voi: `/context all` prima e dopo un `CLAUDE.md` di due righe